

Nama: Akbar Fitri Andhika

NIM: 2410511011

Kelas: Informatika (A)

Dataset

Dataset yang digunakan dalam proyek ini adalah Wine Dataset yang tersedia secara bawaan pada library scikit-learn. Dataset ini merupakan hasil analisis kimia terhadap sampel wine yang berasal dari tiga jenis kultivar (varietas tanaman anggur) berbeda yang tumbuh di wilayah Italia. Dataset ini terdiri dari 178 sampel data dengan 13 fitur numerik sebagai variabel masukan, yaitu:

1. Alcohol : kadar alkohol dalam wine
2. Malic Acid : kadar asam malat
3. Ash : kadar abu
4. Alcalinity of Ash : alkalinitas abu
5. Magnesium : kadar magnesium
6. Total Phenols : total fenol
7. Flavanoids : kadar flavanoid
8. Nonflavanoid Phenols : fenol non-flavanoid
9. Proanthocyanins : kadar proantosianin
10. Color Intensity : intensitas warna
11. Hue : tingkat kejernihan warna (hue)
12. OD280/OD315 of Diluted Wines : rasio absorbansi optik
13. Proline : kadar prolin

Variabel target (label) terdiri dari 3 kelas, yang merepresentasikan tiga jenis kultivar wine berbeda, yaitu Kelas 1 (Wine A), Kelas 2 (Wine B), dan Kelas 3 (Wine C). Dataset ini bersifat publik, bersih (tidak terdapat nilai yang hilang), dan sudah banyak digunakan sebagai benchmark dalam penelitian klasifikasi multikelas.

Pembagian Data

Data dibagi menjadi dua bagian menggunakan fungsi `train_test_split` dari scikit-learn dengan proporsi sebagai berikut:

- Data Latih (Training Set): 80% dari total data, digunakan untuk melatih model

- Data Uji (Testing Set): 20% dari total data, digunakan untuk mengevaluasi performa model

Parameter `random_state=42` digunakan untuk memastikan pembagian data bersifat deterministik dan dapat direproduksi.

Prapemrosesan Data

Sebelum proses pelatihan, seluruh fitur dinormalisasi menggunakan `StandardScaler` dari `scikit-learn`. `StandardScaler` bekerja dengan cara mengubah setiap fitur sehingga memiliki rata-rata (mean) senilai 0 dan standar deviasi senilai 1. Proses ini diperlukan karena algoritma Logistic Regression sensitif terhadap skala fitur yang berbeda-beda.

Objek normalisasi disimpan ke dalam file `normalisasi.pkl` menggunakan `joblib` agar dapat dimuat kembali saat proses inferensi (prediksi), sehingga data masukan baru akan dinormalisasi secara konsisten dengan data latih.

Model

Model yang digunakan adalah Logistic Regression (Regresi Logistik), salah satu algoritma klasifikasi yang umum digunakan untuk masalah klasifikasi multikelas. Meskipun namanya mengandung kata "regresi", algoritma ini digunakan untuk tugas klasifikasi. Konfigurasi model yang digunakan adalah sebagai berikut:

- `max_iter=200` : jumlah iterasi maksimum selama proses optimasi, ditingkatkan dari nilai default (100) untuk memastikan model dapat konvergen
- `random_state=42` : untuk memastikan hasil pelatihan yang konsisten dan dapat direproduksi

Model bekerja dengan cara mempelajari batas keputusan (decision boundary) antarkelas berdasarkan kombinasi linear dari fitur-fitur yang diberikan, kemudian menggunakan fungsi softmax untuk menghasilkan probabilitas keanggotaan terhadap masing-masing kelas.

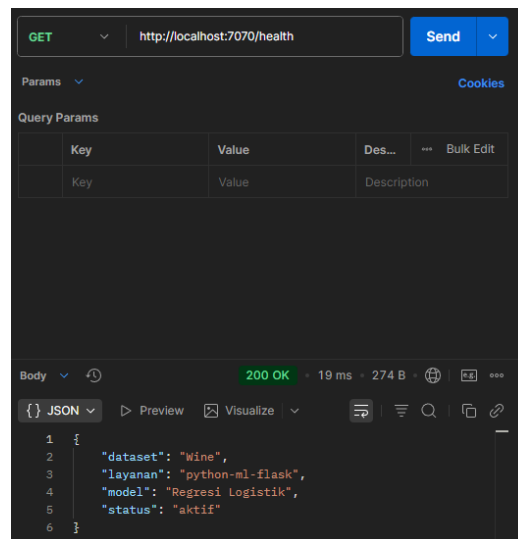
Evaluasi Model

Berdasarkan hasil evaluasi pada data uji, model mencapai akurasi sebesar 100%, yang berarti seluruh sampel pada data uji berhasil diklasifikasikan dengan benar. Performa ini dimungkinkan karena dataset Wine memiliki karakteristik yang cukup terpisah antarkelasnya, sehingga Logistic Regression mampu menemukan batas keputusan yang optimal setelah normalisasi data.

Model yang telah dilatih disimpan ke dalam file `model.pkl` menggunakan `joblib` dan dimuat secara otomatis saat layanan Flask dijalankan.

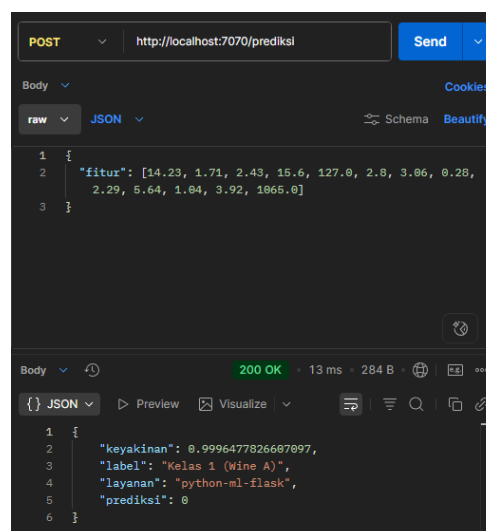
Percobaan Postman

Request: GET <http://localhost:7070/health>



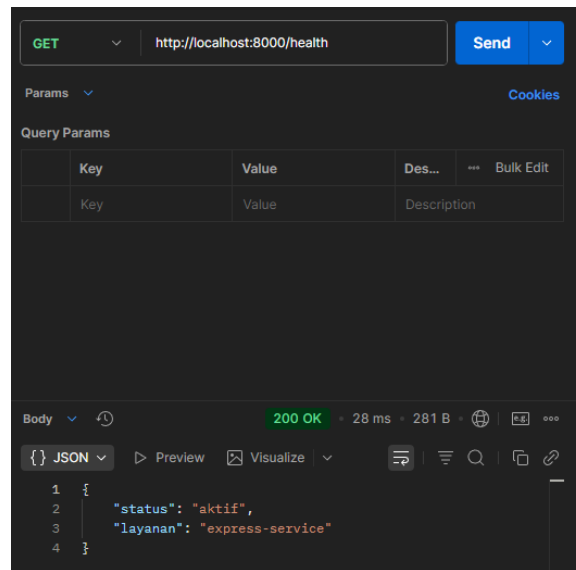
Endpoint ini ditangani langsung oleh layanan Flask tanpa diteruskan ke service manapun. Response yang dihasilkan berupa informasi status layanan, nama layanan, model yang digunakan, dan dataset yang dipakai. Endpoint ini berguna sebagai penanda bahwa layanan Python ML sedang aktif dan siap menerima request prediksi dari client maupun dari Express.

Request: POST <http://localhost:7070/prediksi>



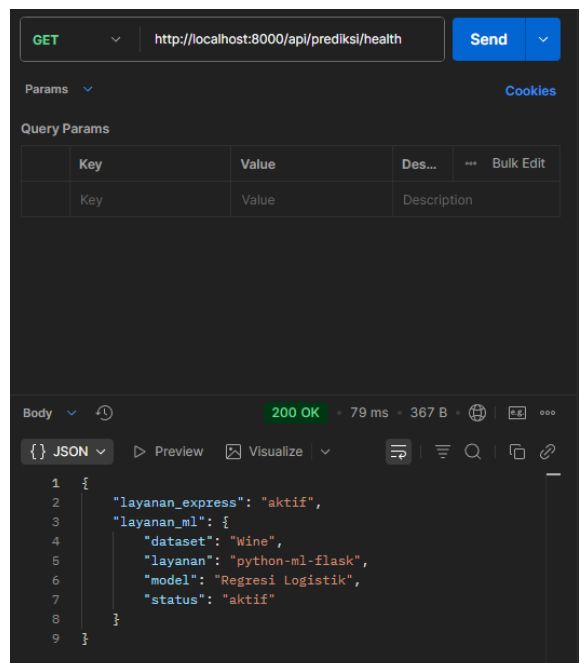
Endpoint ini menerima array berisi 13 fitur wine kemudian memprosesnya melalui model Logistic Regression yang sudah dilatih. Data fitur dinormalisasi terlebih dahulu menggunakan StandardScaler sebelum dimasukkan ke model. Response yang dihasilkan berupa label kelas wine, tingkat keyakinan prediksi, dan nama layanan. Pada percobaan ini fitur yang dikirimkan menghasilkan prediksi Kelas 1 (Wine A) dengan tingkat keyakinan sebesar 0.9997.

Request: GET <http://localhost:8000/health>



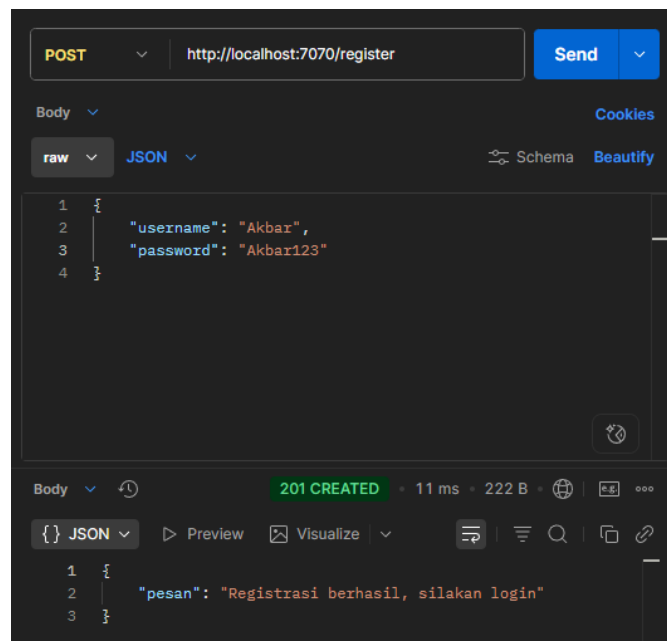
Endpoint ini ditangani langsung oleh layanan Express tanpa diteruskan ke Flask. Response yang dihasilkan berupa status layanan dan nama layanan. Endpoint ini berguna sebagai penanda bahwa layanan Express sebagai gateway sedang aktif dan dapat menerima request dari client.

Request: GET <http://localhost:8000/api/prediksi/health>



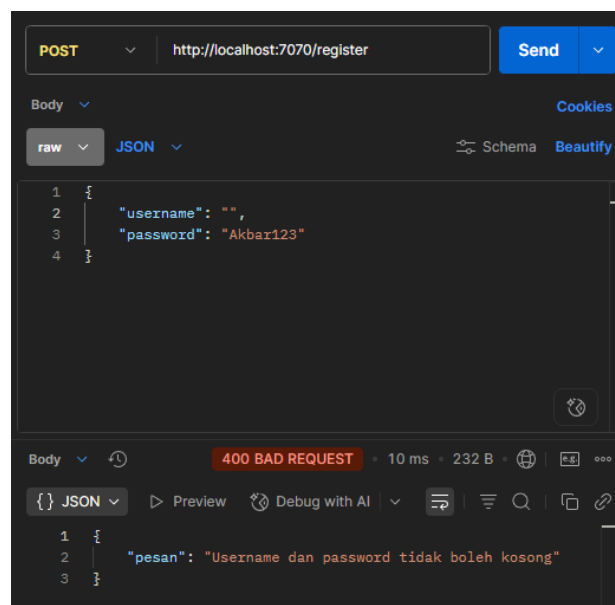
Endpoint ini tidak hanya mengecek status Express saja, tetapi juga sekaligus meneruskan request ke endpoint /health milik Flask untuk mengecek koneksi antar layanan. Apabila Flask aktif, response akan menyertakan informasi lengkap dari Flask di dalam field layanan_ml. Hal ini berguna untuk memastikan kedua layanan berjalan dengan baik sebelum melakukan prediksi.

Request: POST <http://localhost:7070/register>

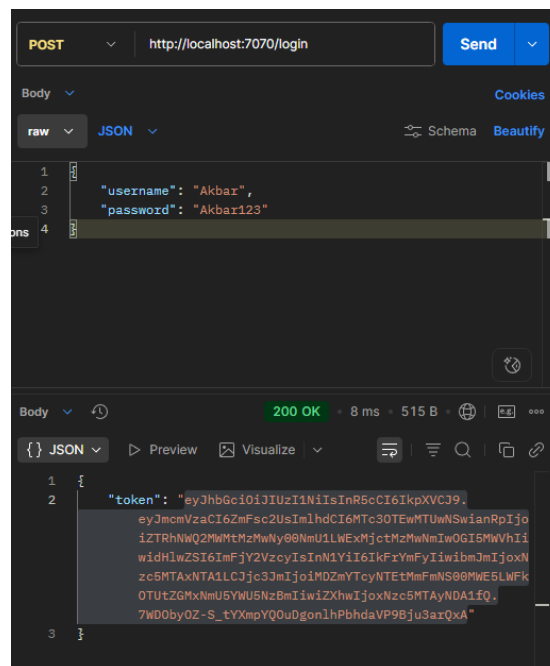


Endpoint ini digunakan untuk mendaftarkan user baru ke dalam sistem. Data user disimpan sementara di memori menggunakan dictionary `users_db`. Pada percobaan ini, registrasi berhasil dilakukan dengan username "Akbar" dan password "Akbar123" sehingga response mengembalikan status 201 Created beserta pesan konfirmasi.

Request: POST <http://localhost:7070/register> dengan username kosong

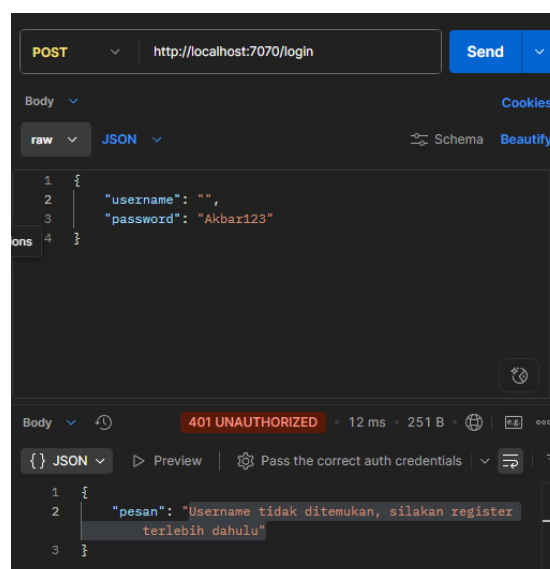


Pada percobaan ini, register dilakukan dengan mengisi username berupa string kosong. Sistem mendeteksi hal tersebut melalui validasi `strip()` dan langsung menolak request dengan response 400 Bad Request beserta pesan yang informatif. Validasi ini penting untuk memastikan data yang tersimpan selalu memiliki nilai yang valid.

Request: POST <http://localhost:7070/login>

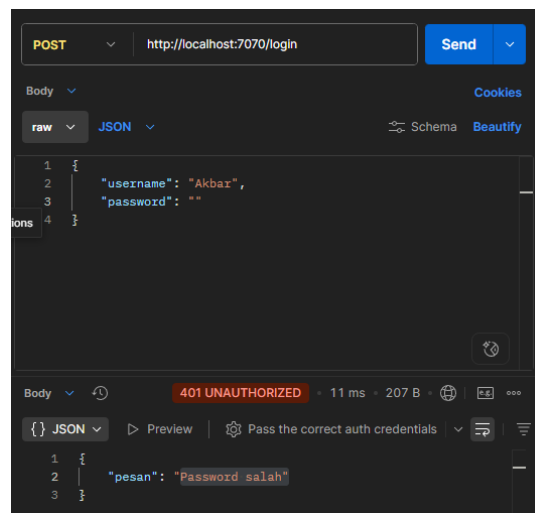
Setelah register berhasil, user melakukan login menggunakan username dan password yang telah didaftarkan. Sistem mencocokkan password yang diinput dengan data yang tersimpan di `users_db`. Apabila cocok, server menerbitkan token JWT yang wajib disertakan sebagai Bearer Token pada setiap request ke endpoint yang membutuhkan autentikasi, yaitu endpoint `/batch-prediksi`.

Request: POST <http://localhost:7070/login> dengan username kosong



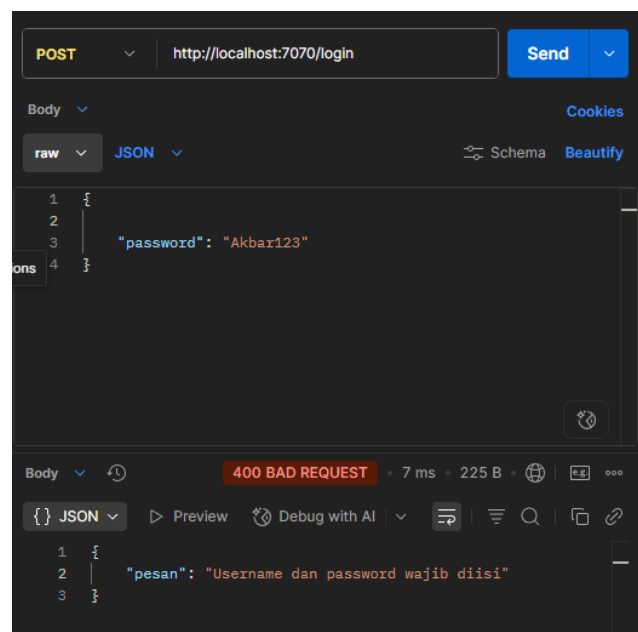
Pada percobaan ini, login dilakukan dengan username yang tidak terdaftar di dalam sistem. Sistem langsung menolak request dengan response 401 Unauthorized beserta pesan yang mengarahkan user untuk melakukan register terlebih dahulu. Mekanisme ini memastikan bahwa hanya user yang sudah terdaftar saja yang dapat memperoleh token.

Request: POST <http://localhost:7070/login> dengan password kosong



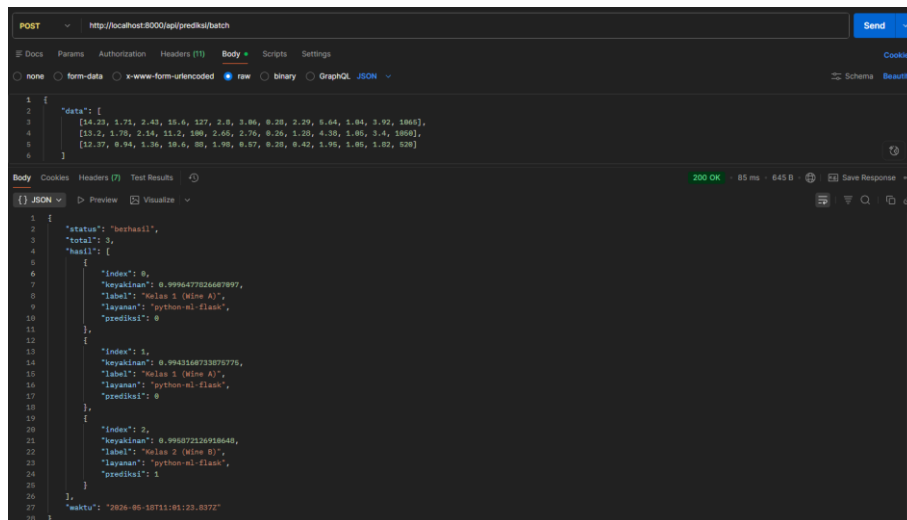
Pada percobaan ini, login dilakukan dengan username yang benar namun password yang salah. Sistem mencocokkan password input dengan data yang tersimpan dan hasilnya tidak sesuai, sehingga request ditolak dengan response 401 Unauthorized beserta pesan "Password salah". Hal ini membuktikan bahwa validasi autentikasi berjalan dengan benar.

Request: POST <http://localhost:7070/login> tanpa field username



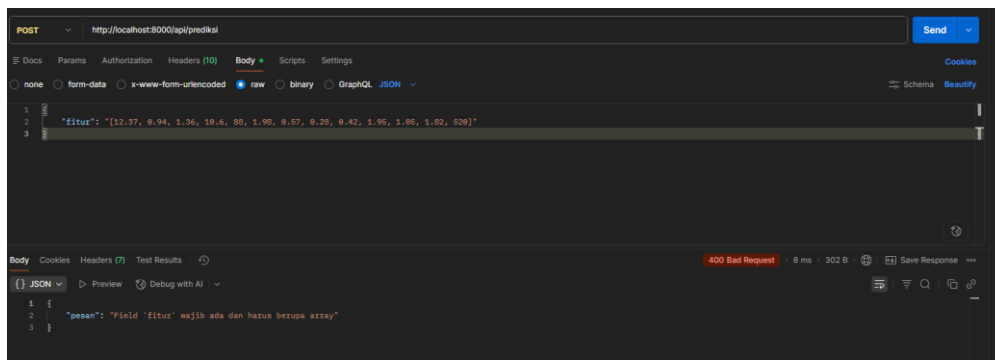
Pada percobaan ini, login dilakukan tanpa menyertakan field username sama sekali di dalam body request. Sistem mendeteksi field yang hilang dan langsung mengembalikan response 400 Bad Request dengan pesan yang informatif. Validasi ini berjalan sebelum proses pencocokan username dan password sehingga lebih efisien.

Request: POST <http://localhost:8000/api/prediksi/batch>



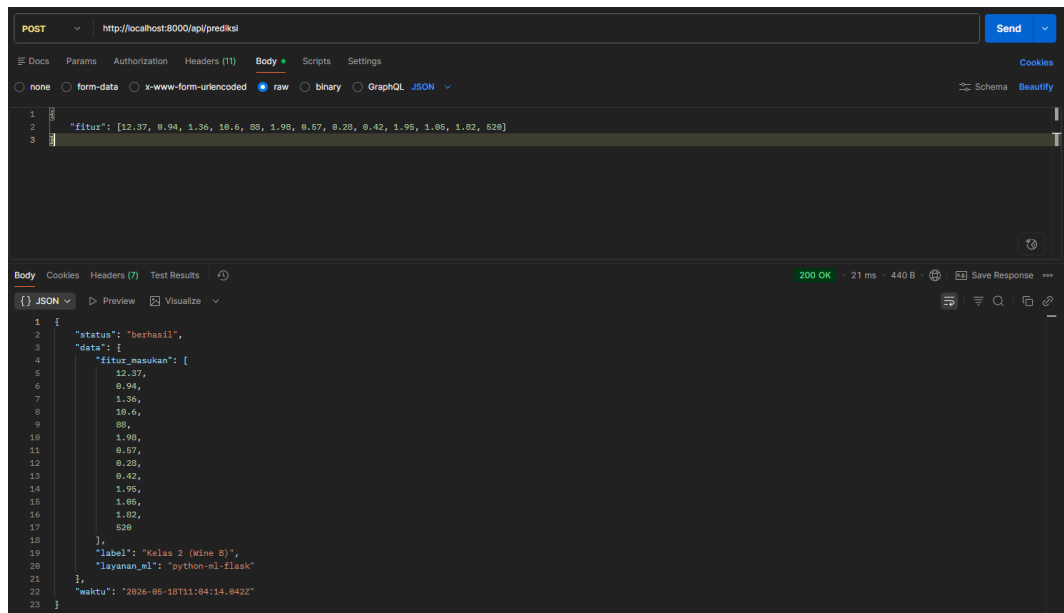
Endpoint ini menerima banyak data fitur sekaligus dalam satu request dan meneruskannya ke endpoint /batch-prediksi milik Flask dengan menyertakan token JWT di header. Pada percobaan ini, tiga data fitur dikirimkan sekaligus dan menghasilkan prediksi Kelas 1 (Wine A) untuk data pertama dan kedua, serta Kelas 2 (Wine B) untuk data ketiga. Response menyertakan waktu pemrosesan dalam format ISO string.

Request: POST <http://localhost:8000/api/prediksi> dengan fitur berupa string



Pada percobaan ini, field fitur dikirimkan dalam bentuk string bukan array. Sistem langsung mendeteksi tipe data yang tidak sesuai menggunakan pengecekan `Array.isArray()` dan menolak request dengan response 400 Bad Request beserta pesan yang informatif. Validasi ini penting untuk mencegah data yang tidak valid masuk ke dalam proses prediksi model.

Request: POST <http://localhost:8000/api/prediksi>



Endpoint ini menerima array fitur dari client dan meneruskannya ke layanan Flask melalui HTTP dengan menyertakan token JWT di header Authorization. Pada percobaan ini, fitur yang dikirimkan menghasilkan prediksi Kelas 2 (Wine B). Response yang dikembalikan Express sudah digabungkan dengan data dari Flask sehingga menyertakan fitur masukan, label prediksi, nama layanan ML, dan timestamp waktu pemrosesan.